

Aprendizaje Maquinal en Pokémon Showdown: Agrupación, Clasificación y Aprendizaje por Refuerzo

Gabriel A. Anzola Tachak
Universidad Nacional de Colombia
Colombia
ga.anzola15@gmail.com

Nicolas David Moreno Villanueva
Universidad Nacional de Colombia
Colombia
nimorenov@unal.edu.co

Abstract—Pokémon Showdown genera millones de replays donde las decisiones de jugadores humanos bajo incertidumbre quedan completamente registradas. Aplicamos tres técnicas de aprendizaje maquinal a 484,130 batallas reales del formato Gen 8 Random Battle para extraer conocimiento de este corpus. Agrupación con K-Means revela dos arquetipos de equipo: ofensivos-rápidos y defensivos-resistentes. Clasificación con XGBoost muestra que la velocidad relativa del equipo es el predictor dominante del estilo de combate ganador (64% de exactitud). Un agente de aprendizaje por refuerzo aprende a ganar el 76.6% de sus batallas por prueba y error, usando las variables que XGBoost identificó como más discriminantes. Las tres técnicas forman un pipeline integrado donde cada etapa transfiere conocimiento a la siguiente.

Index Terms—aprendizaje por refuerzo, gradient boosting, K-Means, Pokémon Showdown, agente autónomo

I. INTRODUCCIÓN

Pokémon es una franquicia de videojuegos de rol creada por Game Freak en 1996, con más de 440 millones de copias vendidas. Su mecánica central son los combates por turnos entre criaturas llamadas Pokémon, cada una con estadísticas base — puntos de vida (HP, del inglés *Hit Points*), Ataque, Defensa, Velocidad —, tipos elementales y hasta cuatro movimientos. Estos combates son estratégicos: cada turno ambos jugadores eligen una acción simultáneamente sin conocer la decisión del rival.

Dos factores dominan la táctica: la **ventaja de tipo** y la **velocidad**. Un movimiento de tipo Agua hace el doble de daño a un Pokémon de tipo Fuego, pero la mitad a uno de Planta — hay dieciocho tipos con relaciones asimétricas entre ellos. La velocidad determina quién actúa primero: el Pokémon más veloz puede noquear al rival antes de recibir contraataque, lo que convierte la composición del equipo en una decisión estratégica de alto impacto.

Pokémon Showdown [1] es un simulador de combate de código abierto con más de 100,000 usuarios activos que replica fielmente estas mecánicas en el navegador. Cada partida queda guardada como un replay público, creando un corpus único: decisiones humanas bajo incertidumbre completamente observables y reproducibles.

El formato **Gen 8 Random Battle** (generación 8, Pokémon Sword/Shield, 2019) asigna equipos aleatorios de seis Pokémon al inicio de cada batalla, eliminando la fase de construcción de equipo y concentrando el desafío exclusiva-

mente en las decisiones durante el combate. La generación 8 introduce además **Dynamax**: un Pokémon puede triplicar temporalmente sus HP durante tres turnos, añadiendo una decisión estratégica adicional.

Este trabajo extrae conocimiento de 484,130 batallas reales [2] con tres técnicas complementarias: (1) K-Means descubre si existen arquetipos naturales de equipo; (2) XGBoost identifica qué características predicen el estilo de combate ganador; (3) un agente de aprendizaje por refuerzo aprende a jugar usando el conocimiento extraído. El código está en <https://github.com/gabotachak/pokemon-agent>.

La contribución central es la *cadena metodológica*: los hallazgos de K-Means y XGBoost informan directamente el diseño del agente, garantizando coherencia entre las tres técnicas sobre el mismo corpus.

II. TRABAJO RELACIONADO

El aprendizaje por refuerzo (RL, del inglés *Reinforcement Learning*) ha sido aplicado a videojuegos desde los años 90, desde programas de Backgammon [3] hasta sistemas de nivel profesional. En el dominio de Pokémon, Sahovic [4] desarrolló poke-env, la librería Python estándar para conectar agentes al simulador Pokémon Showdown, y demostró que agentes basados en redes neuronales profundas (DQN, *Deep Q-Network*) superan el 80% de victorias contra oponentes aleatorios en Gen 1. Jin et al. [5] compilaron PokeChamp, un dataset de millones de batallas con modelos de aprendizaje por imitación — aprender a replicar las acciones de jugadores expertos.

En mayor escala, AlphaStar [6] y OpenAI Five [7] demostraron que RL con redes profundas puede alcanzar nivel profesional en juegos como StarCraft II y Dota 2, aunque con recursos computacionales inaccesibles para la mayoría de contextos de investigación.

A diferencia de estos trabajos, este pipeline combina agrupación no supervisada, clasificación supervisada y RL tabular interpretable en un flujo coherente sobre datos Gen 8, priorizando la extracción de conocimiento comprensible sobre el rendimiento máximo.

III. DATASET Y PREPROCESAMIENTO

A. Dataset

El corpus HolidayOugi/pokemon-showdown-replays [2] contiene 31.7 millones de replays. Se seleccionó el formato gen8randombattle (**484,130 batallas**), descargando únicamente los 3 archivos parquet correspondientes para evitar la descarga completa de 66 GB.

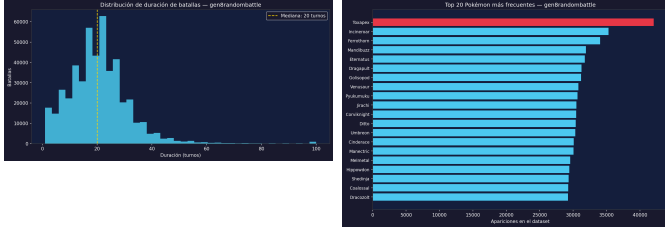


Fig. 1: Izq.: duración de batallas en turnos (mayoría entre 15 y 40). Der.: Pokémon más frecuentes — reflejan el pool aleatorio del formato.

B. Parsing y Feature Engineering

El campo log de cada batalla codifica cada evento en líneas del tipo `|COMANDO|argumentos`. Se extrae: los equipos revelados, el ganador, duración, movimientos usados y cambios voluntarios. Los nombres de Pokémon se normalizan y combinan con estadísticas base [8] para calcular promedios por equipo.

TABLE I: FEATURES DERIVADAS CON SU INTERPRETACIÓN EN CONTEXTO DE COMBATE (SELECCIÓN).

Feature	Interpretación táctica
avg_speed	Velocidad promedio — el Pokémon más veloz actúa primero cada turno
speed_advantage_ratio	Velocidad p1 / p2 — quién controla el ritmo del combate
stat_total_diff	Diferencia de stats totales — proxy de ventaja estadística global
type_coverage	Tipos de movimientos disponibles — amplitud ofensiva del equipo
switch_rate	Cambios por turno — estilo agresivo (bajo) vs. defensivo (alto)
n_fast_pokemon	Pokémon con velocidad >100 — capaces de actuar antes que la mayoría

La **variable objetivo** para clasificación es `winning_action_type`: el tipo de acción más frecuente del jugador ganador. Responde a *¿cómo ganan los que ganan?* Cuatro categorías: movimiento físico, especial, de estado o cambio de Pokémon.

IV. METODOLOGÍA

A. Agrupación — K-Means

¿Existen estilos de equipo naturales en batallas aleatorias, o todos los equipos son estadísticamente indistinguibles?

K-Means [9] es un algoritmo de agrupamiento no supervisado que asigna cada punto al grupo más cercano y recalcula los centros iterativamente hasta convergencia, minimizando la varianza interna de cada grupo:

$$J = \sum_k \sum_{x \in C_k} \|x - \mu_k\|^2 \quad (1)$$

donde μ_k es el centro del grupo k . No requiere etiquetas previas — descubre la estructura que emerge naturalmente de los datos.

Para elegir el número de grupos K , usamos el *silhouette score*: mide qué tan bien separado está cada punto de los demás grupos. Se calcula comparando la cohesión interna con la distancia al grupo vecino más cercano:

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}, \quad s(i) \in [-1, 1] \quad (2)$$

donde $a(i)$ es la distancia media a los otros puntos del mismo grupo y $b(i)$ la distancia media al grupo vecino más cercano. Valores cercanos a 1 indican grupos bien separados; valores cercanos a 0 indican solapamiento.

Pipeline: vectores de equipo (9 features) → normalización → análisis de componentes principales (PCA, para reducir a 2 dimensiones y poder visualizar) → K-Means $K=2..10$.

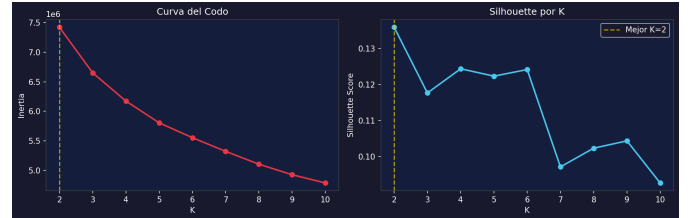


Fig. 2: Inercia y silhouette score para $K=2..10$. El máximo de silhouette ocurre en $K=2$.

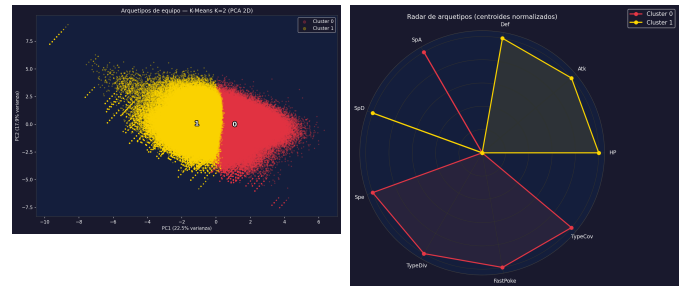


Fig. 3: Proyección PCA coloreada por grupo (izq.) y radar de centroides (der.). Grupo 0: mayor velocidad. Grupo 1: mayor HP y defensa.

Resultado: $K=2$ con $\text{silhouette}=0.136$. El bajo valor es inherente al formato aleatorio — todos los equipos provienen del mismo pool. Aun así, los centroides revelan dos estilos: equipos veloces y ofensivos vs. equipos resistentes y defensivos.

B. Clasificación — XGBoost

¿Dado el perfil estadístico de ambos equipos antes de la batalla, qué estrategia llevará al ganador a la victoria?

XGBoost [10] implementa *gradient boosting*: construye un conjunto (*ensemble*) de árboles de decisión de forma aditiva,

donde cada árbol nuevo corrige los errores del anterior. El modelo final es la suma de T árboles:

$$\hat{y} = \sum_{t=1}^T f_t(x) \quad (3)$$

En cada iteración, se elige el árbol f_t que minimiza la pérdida acumulada más un término $\Omega(f_t)$ que penaliza árboles innecesariamente complejos para evitar sobreajuste:

$$\mathcal{L}^{(t)} = \sum_i \ell(y_i, \hat{y}_i^{(t-1)} + f_t(x_i)) + \Omega(f_t) \quad (4)$$

Además de predicciones, XGBoost cuantifica la **importancia** de cada feature: cuánto contribuyó cada variable al ensemble.

Configuración: 22 features, split 80/20 estratificado, 1000 árboles máximo con parada temprana a los 20 sin mejora, GPU. Mejor iteración: 820 árboles.

TABLE II: MÉTRICAS POR CLASE. EXACTITUD GLOBAL: 64.0% (BASELINE: 54%). EL ÁREA BAJO LA CURVA ROC (AUC-ROC) MIDE LA CAPACIDAD DISCRIMINATIVA: 1.0 ES PERFECTO, 0.5 EQUIVALE A CLASIFICACIÓN ALEATORIA.

Clase	Precisión	Recall	F1	AUC-ROC
physical	0.672	0.871	0.758	0.786
special	0.506	0.228	0.315	0.766
status	0.496	0.304	0.377	0.798
switch	0.660	0.640	0.650	0.955
promedio			0.606	0.826



Fig. 4: Importancia de features por contribución al ensemble (izq.) y matriz de confusión normalizada (der.)

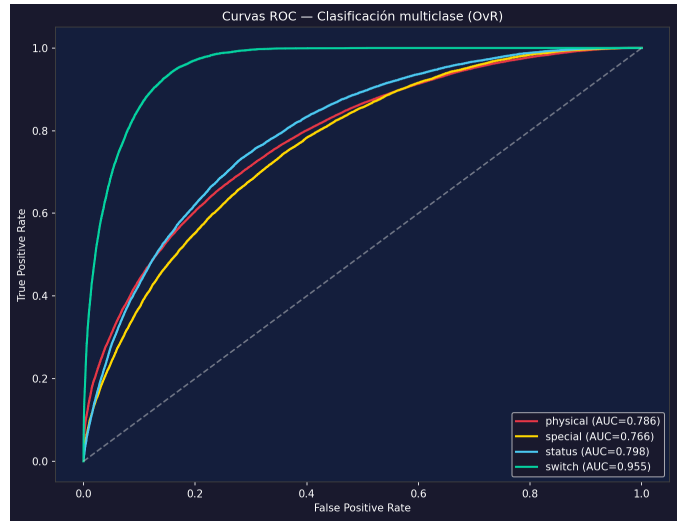


Fig. 5: Curvas ROC por clase. El AUC-ROC de switch (0.955) destaca: cuando el ganador cambia frecuentemente de Pokémon, el patrón estadístico es inequívoco.

La feature más importante es `speed_advantage_ratio`: en Gen 8, actuar primero puede significar noquear al rival sin recibir daño. El bajo F1 en special y status refleja desbalance — physical representa el 54% de los registros.

C. Aprendizaje por Refuerzo — Q-Learning

¿Puede un agente aprender a jugar Pokémon Show-down por prueba y error, sin que se le diga explícitamente qué hacer?

El aprendizaje por refuerzo [11] entrena un agente que interactúa con un entorno: toma acciones, recibe recompensas y ajusta su comportamiento para maximizar la recompensa acumulada a largo plazo. No requiere ejemplos etiquetados ni reglas predefinidas — el agente descubre qué funciona por experiencia directa.

Q-Learning [12] es uno de los algoritmos más simples de RL. Aprende una tabla $Q(s, a)$ que estima la recompensa futura esperada de tomar acción a en estado s . Tras cada turno, el agente actualiza su estimación:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right] \quad (5)$$

El término $r + \gamma \max_{a'} Q(s', a')$ combina la recompensa inmediata r con el mejor valor futuro conocido desde el nuevo estado s' , descontado por $\gamma = 0.9$. La diferencia con $Q(s, a)$ es el error de predicción que el agente corrige con tasa $\alpha = 0.1$.

Conexión con el corpus: el espacio de estados se diseñó a partir de los análisis anteriores, no arbitrariamente:

TABLE III: VARIABLES DE ESTADO DEL AGENTE Y SU ORIGEN EN EL ANÁLISIS PREVIO.

Variable	Valores	Origen en el corpus
hp_self/opp	4 rangos	Distribución de HP en 484k batallas
type_advantage	{-1,0,+1}	Feature discriminante en XGBoost
can_outspeed	booleano	Derivada de speed_advantage_ratio — top feature
team_size_self/opp	1-6	Tamaño de equipo diferencia arquetipos K-Means
has_switch	booleano	switch_rate fue feature relevante en el corpus

El agente juega contra un **RandomPlayer** — oponente que elige acciones uniformemente al azar, sin considerar el estado del combate. Sirve como línea base para verificar que el agente aprende algo.

Espacio de acciones: 9 fijas — slots 0-3 son movimientos, slots 4-8 son cambios de Pokémon. Acciones inválidas caen al primer movimiento disponible.

Señal de recompensa: +1.5 por daño infligido, -1.0 por daño recibido sin contraatacar, ± 0.5 por calidad del cambio, ± 10 por victoria o derrota al terminar la batalla.

Exploración ϵ -greedy: el agente comienza eligiendo al azar ($\epsilon=1.0$) y decae hasta $\epsilon=0.05$ a lo largo de 1,500 de los 2,000 episodios totales.

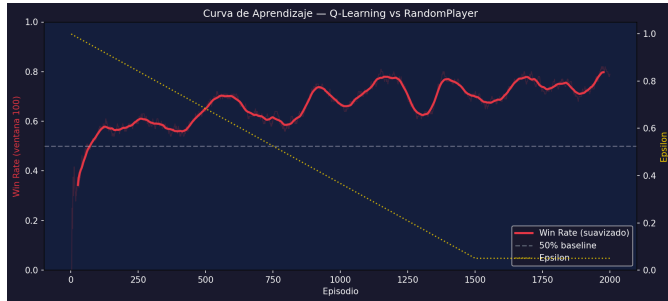


Fig. 6: Win rate promedio (ventana de 100 episodios) y decaimiento de ϵ durante el entrenamiento.

VI. RESULTADOS Y DISCUSIÓN

K-Means: $K=2$, silhouette=0.136. PCA captura 40.4% de varianza en 2 dimensiones. Los centroides son interpretables como estilos ofensivo y defensivo débilmente separados, coherente con equipos de un pool aleatorio común.

XGBoost: 64.0% de exactitud, +10 puntos sobre el baseline. speed_advantage_ratio lidera la importancia: quien actúa primero controla el combate. El F1 bajo en special (0.315) y status (0.377) refleja el desbalance de clases — physical domina el 54% de los registros. El AUC-ROC de switch (0.955) revela que los cambios de Pokémon son la señal estadística más discriminante: cuando el ganador cambia frecuentemente, el patrón es claro e inequívoco.

Q-Learning: el agente pasa de 46% de victorias (nivel aleatorio) a 76.6% en 2,000 episodios. Solo visita 2,039 de los 27,648 estados teóricos posibles (7%) — una ventaja del enfoque tabular: el agente aprende únicamente sobre

situaciones reales, sin gastar capacidad en combinaciones imposibles. La tabla Q es completamente inspeccionable: a diferencia de una red neuronal, se puede leer directamente qué acción prefiere el agente en cada situación concreta.

Cadena metodológica: speed_advantage_ratio (XGBoost) $\rightarrow$ can_outspeed (estado del agente). Arquetipos K-Means $\rightarrow$ team_size en el estado. Ninguna técnica opera en aislamiento.

Limitaciones: el 76.6% es contra RandomPlayer y no generaliza a oponentes más fuertes. Q-Learning tabular no escala a estados continuos.

Comparativa de las 3 Técnicas de ML			
Técnica	K-Means	XGBoost	Q-Learning
Tipo	Agrupación	Clasificación	Refuerzo
Datos	484k equipos	484k batallas	2000 episodios
Métrica principal	Silhouette=0.136	Accuracy=64.0%	Win Rate=76.6%
Resultado	K=2 clusters	F1=0.606	2039 Q-states
Device	CPU	CUDA	CPU

Fig. 7: Comparativa de las tres técnicas.

VI. CONCLUSIONES

- 1) **K-Means** encontró 2 arquetipos de equipo (silhouette=0.136), con centroides interpretables como estilos ofensivo y defensivo.
- 2) **XGBoost** predijo el estilo de combate ganador con 64.0% de exactitud, confirmando que la velocidad relativa es el factor dominante en Gen 8.
- 3) **El agente de aprendizaje por refuerzo** alcanzó 76.6% de victorias con una tabla Q de 2,039 estados diseñada a partir de los hallazgos anteriores.

La cadena K-Means $\rightarrow$ XGBoost $\rightarrow$ Q-Learning demuestra que la agrupación no supervisada y la clasificación supervisada pueden informar directamente el diseño de un agente de refuerzo. La tabla Q inspeccionable permite verificar comportamientos tácticamente coherentes: cambiar de Pokémon ante desventaja de tipo, atacar agresivamente cuando se puede actuar primero.

Trabajo futuro: reemplazar la tabla Q por un DQN para manejar estados continuos, entrenar contra un oponente más fuerte, y analizar la tabla Q para extraer reglas de juego en lenguaje natural. Código: <https://github.com/gabotachak/pokemon-agent>.

REFERENCES

- [1] Smogon University, “Pokémon Showdown Battle Simulator.” [Online]. Available: <https://github.com/smogon/pokemon-showdown>
- [2] HolidayOugi, “Pokémon Showdown Replays Dataset.” [Online]. Available: <https://huggingface.co/datasets/HolidayOugi/pokemon-showdown-replays>
- [3] G. Tesauro, “TD-Gammon, a Self-Teaching Backgammon Program,” *Machine Learning*, vol. 14, pp. 397–422, 1994.
- [4] H. Sahovic, “poke-env: A Python Interface for Pokémon Showdown.” [Online]. Available: <https://github.com/hsahovic/poke-env>
- [5] Y. Jin and others, “PokeChamp: an Expert-level Minimax Language Agent for Competitive Pokémon,” *arXiv:2506.04765*, 2025.
- [6] O. Vinyals and others, “Grandmaster Level in StarCraft II Using Multi-Agent Reinforcement Learning,” *Nature*, vol. 575, pp. 350–354, 2019.
- [7] C. Berner and others, “Dota 2 with Large Scale Deep Reinforcement Learning,” technical report arXiv:1912.06680, 2019.

- [8] PokeAPI, “The RESTful Pokémon API.” [Online]. Available: <https://pokeapi.co/>
- [9] J. MacQueen, “Some Methods for Classification and Analysis of Multivariate Observations,” in *Proc. 5th Berkeley Symp. Mathematical Statistics and Probability*, 1967, pp. 281–297.
- [10] T. Chen and C. Guestrin, “XGBoost: A Scalable Tree Boosting System,” in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016, pp. 785–794. doi: 10.1145/2939672.2939785.
- [11] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018.
- [12] C. J. C. H. Watkins and P. Dayan, “Q-Learning,” *Machine Learning*, vol. 8, pp. 279–292, 1992.